

Audio Transcription AI Agent

1. Introduction

This project presents a complete AI-based system that converts audio input into text and processes it intelligently based on user requirements. The system uses the Whisper model for speech-to-text conversion and integrates a Large Language Model (Gemini) for generating summaries and extracting keywords.

The application supports multilingual audio and provides an option for translation into English. It includes robust error handling, performance optimization, and a user-friendly interface built using Streamlit.

The system is designed as a lightweight, terminal + UI-based application that demonstrates the full lifecycle of an AI pipeline including data processing, intelligent routing, robustness, and performance optimization.

2. Objective

The main objectives of this project are:

- Convert audio input into text using AI
 - Process the text based on user intent
 - Generate summaries and keywords using LLM
 - Handle real-world errors and edge cases
 - Measure performance and optimize system
 - Provide a user-friendly interface
-

3. Methodology

The project follows a step-by-step pipeline:

1. **Audio Input**

User uploads an audio file (.mp3 or .wav)

2. **Speech-to-Text Conversion**

Whisper model converts audio into text

3. **Text Preprocessing**

Cleaning and formatting of text

4. **Validation Layer**

Checks for empty, noisy, or invalid input

5. **Intent Routing**

User selects required operation:

- Transcript
- Summary
- Keywords

6. **AI Processing**

Gemini API processes text for summary/keywords

7. **Output Generation**

Displays result along with metrics

8. **Logging**

Stores execution details for analysis

4. System Architecture

Audio Input



Whisper Model (Speech-to-Text)



Text Cleaning



Validation Layer (Error Handling)



Intent Routing (User Selection)



Gemini API (LLM Processing)



Output + Metrics + Logs

5. Technologies Used

- Python
 - OpenAI Whisper (Speech-to-Text)
 - Google Gemini API (LLM)
 - Streamlit (Frontend UI)
 - dotenv (Environment Variables)
 - FFmpeg (Audio Processing)
-

5. Features

Audio Processing

- Supports .mp3 and .wav files
 - Converts speech to text
 - Supports multilingual audio
-

Intelligent Processing

- Full Transcript
 - Summary generation
 - Keyword extraction
-

Translation Feature

- User can choose to translate audio into English
 - Maintains flexibility and control
-

Error Handling (Day 13)

- Empty input detection
 - Invalid file format handling
 - Noise / short audio filtering
 - File path validation
 - Gemini failure fallback
-

Performance Optimization (Day 14)

- Transcription latency measurement

- Gemini response time tracking
 - Token estimation
 - Prompt optimization
-

User Interface (Day 15)

- Built using Streamlit
 - Upload audio file
 - Audio preview player
 - Task selection
 - Real-time output display
 - Download output feature
-

Logging System

- Logs stored in logs/log.txt
 - Records:
 - File name
 - Task
 - Tokens
 - Processing time
 - Output preview
-

7. Testing & Evaluation

The system was tested using multiple real-world scenarios:

Test Case	Result
Normal Audio	Success
Empty Input	Handled
Noise Audio	Rejected
Long Audio	Trimmed
Invalid File	Handled

8. Performance Metrics

The system tracks:

- Transcription Time
- Gemini Response Time
- Approx Token Count

Example:

Transcription Time: 3.12 sec

Gemini Time: 1.05 sec

Approx Tokens: 120

9. Challenges Faced

- FFmpeg not detected initially
- File path errors across folders
- Gemini API key validation issues
- Whisper model download corruption
- Handling noisy and empty audio
- Managing large input size

10. Solutions Implemented

- Configured FFmpeg in system PATH
 - Implemented input validation checks
 - Used dotenv for secure API key handling
 - Added fallback for API failures
 - Limited input size to improve performance
 - Added logging system for debugging
-

11. Conclusion

This project successfully demonstrates a complete AI pipeline, including:

- Data ingestion (audio processing)
- Intelligent decision-making
- Robust error handling
- Performance optimization
- User-friendly interface

It reflects real-world system design and practical AI implementation.

12. Conclusion

The project successfully demonstrates the implementation of an AI-powered audio processing system. It combines speech recognition and natural language processing with a user-friendly interface, making it a practical and efficient solution for audio transcription and analysis.